

To

Delft3D users

Date

2026-07-17

Our reference

001

Number of pages

4

Contact person

Johan Reyns

Direct line**E-mail**

johan.reyns@deltares.nl

Subject

Slurm MPI Orchestration for MorMerge

1 Purpose

This note documents the Slurm orchestration path added to `mormerge.tcl` for MorMerge runs that use MPI-partitioned models. The implementation keeps the existing local, SGE, and Torque behavior intact, but removes the previous Slurm restriction that only allowed `nodes = 1`.

The Slurm path now uses one Slurm allocation for the complete MorMerge run. Within that allocation, the Tcl driver derives a resource geometry for each condition, starts the serial MorMerge helper processes from the batch script, and starts each DIMR condition through an explicit exclusive `srun` MPI job step. Slurm selects the physical nodes for those job steps; MorMerge does not pin a condition to a manually constructed node list.

2 User-facing configuration

The existing `nodes`, `queueSystem`, `queue`, and `timeLimit` keys are still used. The Slurm path adds the following optional keys to the `.mm` file:

slurmControlNodes Capacity excluded from the condition-step geometry before condition resources are derived. This is not a Slurm resource reservation and does not select or isolate a physical control node; Slurm may place condition steps on any node in the allocation. Default: 0.

slurmNodesPerCondition Nodes assigned to each condition. Default: 0 for automatic equal partitioning.

slurmTasksPerNode MPI ranks per node for DIMR condition steps. Default: 1.

slurmCpusPerTask CPU threads per MPI rank. Default: 1.

slurmMPI Optional `srun --mpi` value, for example `pmix` or `pmi2`. Use `none` to omit.

slurmModuleCommand Module command emitted in the generated `sbatch` script. Default: `none`. Use a site-specific command only when the selected Delft3D installation requires it.

Example for two MorMerge conditions, each running DIMR on two nodes with 32 MPI ranks per node, plus one control node:

```
queueSystem          = slurm
```

```
queue                = normal
timeLimit            = 02:00:00
nodes                = 5

dimrexedir           = /opt/delft3dfm/lnx64/bin
dimrargs             = dimr_config.xml
mormergeexedir       = /opt/delft3dfm/lnx64/bin

slurmControlNodes    = 1
slurmNodesPerCondition = 2
slurmTasksPerNode    = 32
slurmCpusPerTask     = 1
slurmMPI             = pmix
slurmModuleCommand   = module load <site-mpi-module>

condition:weight     = storm_a : 0.5
condition:weight     = storm_b : 0.5
```

3 Generated Slurm allocation

When the Tcl driver is started outside an allocation and `queueSystem = slurm`, it generates and submits `d3d-mormerge_qsubgo.sh` using `sbatch`. The generated script now contains explicit Slurm resources:

```
#SBATCH --job-name=mormerge
#SBATCH --nodes=<nodes>
#SBATCH --ntasks-per-node=<slurmTasksPerNode>
#SBATCH --cpus-per-task=<slurmCpusPerTask>
#SBATCH --time=<timeLimit>
#SBATCH --partition=<queue>
#SBATCH --output=d3d-mormerge-%j.out
#SBATCH --error=d3d-mormerge-%j.err
```

It exports the configured `slurmCpusPerTask` value to the common OpenMP and threaded-library variables, prints basic job metadata and the expanded `SLURM_JOB_NODELIST` for diagnostics, and then re-enters `mormerge.tcl` with the internal `-n` node count. The generated script does not impose an MPI implementation, fabric provider, or CPU-binding policy.

4 Condition-step geometry

Inside the allocation, `mormerge.tcl` validates the requested capacity:

```
required nodes =
    slurmControlNodes
    + number_of_conditions * slurmNodesPerCondition
```

If `slurmNodesPerCondition = 0`, the condition nodes are derived automatically by evenly dividing the non-control capacity over the conditions. If the division is ambiguous, the script fails early and asks for an explicit `slurmNodesPerCondition` value. The legacy one-node Slurm case is still allowed; exclusive condition steps wait for the allocation rather

than oversubscribing it. For multi-node jobs, `--exclusive` ensures that concurrent condition steps receive disjoint Slurm resources without a hard-coded `--odelist`.

5 Launch model

The launch order remains unchanged:

- 1 clone input into one condition directory per condition;
- 2 clear merge/sync;
- 3 start one MorMerge helper process per run id;
- 4 wait for each merge<runid> sync marker;
- 5 start one model process per condition;
- 6 wait for condition sync markers and for launched processes to finish.

For Slurm, the MorMerge helper scripts are run as ordinary serial processes from the batch script. This avoids turning the helper into an MPI job step and preserves the existing stream-file protocol.

For DIMR conditions, `dimrexedir` resolves to `run_dimr.sh`. The condition launcher performs directory setup once, then starts that wrapper with an explicit job step:

```
srun --exclusive --kill-on-bad-exit=1 [--mpi=<slurmMPI>] \  
  --nodes=<slurmNodesPerCondition> \  
  --ntasks=<slurmNodesPerCondition * slurmTasksPerNode> \  
  --ntasks-per-node=<slurmTasksPerNode> \  
  --cpus-per-task=<slurmCpusPerTask> \  
  --export=ALL,OMP_NUM_THREADS=<slurmCpusPerTask>,... \  
  "<dimrexedir>/run_dimr.sh" <dimrargs>
```

This contract requires `run_dimr.sh` to be a passive wrapper: it may set the Delft3D runtime environment, but it must not invoke `srun`, `mpirun`, or another MPI launcher. MorMerge owns the single outer `srun` step. An MPI-owning site wrapper must instead be configured as a custom `runscript` and must use the exported per-condition Slurm geometry.

The generated condition script records each background child PID, waits for each PID separately, and returns a nonzero status if any launcher fails. This keeps a failed `srun` step from being reported later as only a missing synchronization marker.

Direct Delft3D-FLOW execution through `flowexedir` is intentionally limited to one Slurm task per condition, because launching a serial FLOW executable with multiple `srun` tasks would start multiple copies. Use `dimrexedir` for MPI-partitioned DIMR runs. Custom `runscript` entries are not wrapped in `srun`. A custom script owns its MPI launch and may use the site-specific `srun`, `mpirun`, or equivalent launcher required by that cluster. MorMerge exports the per-condition values `MORMERGE_SLURM_NODES`, `MORMERGE_SLURM_TASKS`, `MORMERGE_SLURM_TASKS_PER_NODE`, and `MORMERGE_SLURM_CPUS_PER_TASK`. A custom Slurm launcher must use these values when creating its exclusive `srun` step, rather than inheriting the complete allocation geometry.

6 Wrapper change

`run_mormerge.sh` now forwards `--NNODES <nodelist>` to `mormerge.tcl` as the internal `-n <nodelist>` argument. This makes the wrapper usable when a site script has already created the Slurm allocation and only wants the Tcl driver to use the existing node count.

7 Operational notes

Use `slurmControlNodes` only to leave condition-step capacity unused when deriving the concurrent geometry. It does not reserve a node or isolate the MorMerge controller. Set it to 0 when no unused capacity is required. Keep `slurmTasksPerNode` aligned with the MPI decomposition in the DIMR configuration, and keep `slurmCpusPerTask` aligned with the intended OpenMP or threaded-library count per rank.

Set `slurmModuleCommand` and `slurmMPI` only to values supported by the target cluster and compatible with the Delft3D installation. The defaults make no Intel MPI or network-fabric assumption.

The generated Slurm scripts are retained when `debug = 1` in the `.mm` file. That is the recommended mode for first runs on a new cluster, because it leaves the generated `sbatch`, condition launcher, and MorMerge launcher scripts available for inspection.